Pick up and Place with Calibrated Visual Servoing

Ray Shu
Department of Computing Science
University of Alberta
Edmonton, Canada
rsshu@ualberta.ca

***Abstract*—We present an integrated robotic manipulation system that combines instance segmentation, image-moment-based visual servoing, and ROS-based robot control to detect, align with, grasp, and drop geometric shapes. The perception pipeline was built around a YOLO segmentation model trained on a custom synthetic dataset of geometric shapes, enabling the system to extract binary masks for objects such as circles, hexagons, pentagons, and crosses. From these masks, the system computes image moments to estimate object centroid points and area, which are then compared against stored target features to send velocity commands to the Kinova. A ZeroMQ interface connects the vision process to a Kinova Gen3 control stack running in ROS, allowing the robot to request velocity updates, transition from tracking to grasp execution, and coordinate predefined grasp motions once alignment thresholds are met. The system achieved a 9/10 success rate on the full pick-and-place task, completing each trial in approximately 1 minute and 40 seconds.**

***Index Terms*—visual servoing, robot manipulation, instance segmentation**

I. Introduction

Robotic manipulation in unstructured environments depends on a robot's ability to perceive objects reliably and adjust its motion continuously as the scene changes. This project implements a vision-guided manipulation system that combines modern instance segmentation with classical image-based visual servoing to enable a Kinova Gen3 robot arm to detect geometric shapes, align to them, grasp them, and search for a target container for placement. In the implemented pipeline, a YOLO segmentation model identifies geometric objects from camera images, binary masks are converted into image-moment features such as centroid and area, and those features are used to compute closed-loop velocity commands to the Kinova. A ROS-based control stack handles robot motion and gripper execution, while ZeroMQ communication synchronizes the vision node and robot controller during tracking, alignment, grasping, and perimeter search behaviors.

This problem is interesting because it sits at the intersection of computer vision, robotics, and real-time control. Learned perception methods such as YOLO are effective at separating objects from cluttered backgrounds, but perception alone does not solve manipulation. A robot must also decide how visual error should translate into physical motion, especially when precise alignment is required for grasping. This project is therefore motivated by the need to bridge high-level visual recognition and low-level robot control in a practical, end-to-end system. Rather than treating object detection and robot execution as separate tasks, the project integrates them into a single feedback loop in which segmentation outputs directly drive servoing behavior. Unlike fixed industrial manipulation systems that rely on predetermined object positions, this system handles objects at arbitrary locations within the workspace, adapting its behavior through continuous visual feedback. The source code for the project can be found on GitHub.[1]

II. Related Work

A. Image-Based Visual Servoing

This project is primarily related to work in image-based visual servoing, instance segmentation, and vision-guided robotic manipulation. Image-based visual servoing uses visual features measured in the image plane to control robot motion directly, making it well suited for real-time alignment tasks [1]. More specifically, moment-based visual servoing uses features such as centroid, area, and orientation [2]. This project uses centroid and area as the primary visual features for computing alignment error and generating control commands.

B. Deep Instance Segmentation for Perception

The perception pipeline is also connected to recent work on deep instance segmentation for robotics. Models such as YOLO make it possible to detect and segment objects efficiently, while segmentation masks provide more precise geometric information than bounding boxes alone [3]. In this project, segmented masks are used not only for recognition but also for contour extraction, grasp-related measurements, and visual servoing.

C. Modular Vision-Guided Manipulation Systems

The overall system is similar to modular robotic manipulation pipelines that divide the task into stages such as detection, alignment, grasping, and placement [4]. Rather than relying on a fully end-to-end learned policy, this project combines learned perception with structured feedback control, which is a common and practical design choice in robotic manipulation systems.

[1] https://github.com/Ray-Shu/Robot-Pick-and-Place

III. Methods

The implemented system combines learned instance segmentation, moment-based image features, and closed-loop robot control for object pickup and placement. The manipulation platform was a 7-DoF Kinova Gen3 arm with a Robotiq 2F-85 gripper, driven through the Kortex ROS interface and Cartesian twist commands [5]. The vision process ran in Python and used Ultralytics YOLO for segmentation, OpenCV for mask processing and moment computation, and ZeroMQ for communication between the perception and robot-control processes [3], [6]. The runtime environment was organized around ROS Noetic in Docker, with GPU-enabled inference support for the segmentation model.

A. Perception Pipeline

The object detector was a YOLO11 segmentation model trained on a synthetic dataset of tabletop geometric objects generated with randomized scale, pose, and background appearance (shown in Fig. 1). The model was trained on 10,000 samples for 50 epochs, with color and rotation augmentations applied.

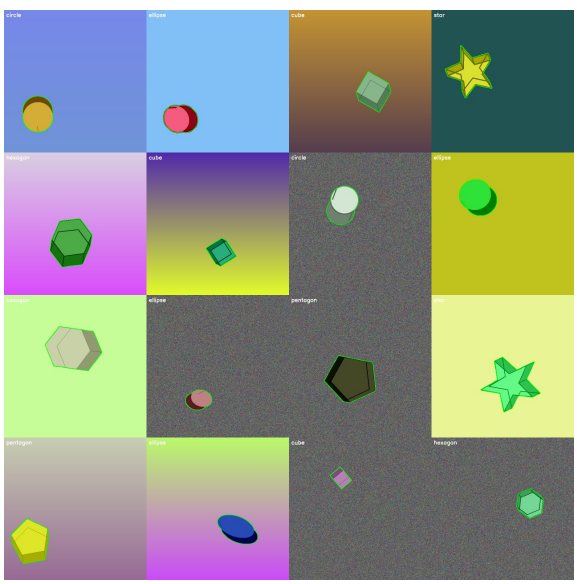

Fig. 1. A set of synthetic 3D shapes.

Each training label was stored as a polygonal segmentation mask rather than a bounding box so that the downstream controller could use the full object silhouette. The trained model was executed on 640×480 RGB frames, and the highest-confidence mask was converted to a binary image for geometric feature extraction.

Let the segmented region be denoted by Ω. Spatial image moments were computed as

$$m_{pq} = \sum_{(u,v)\in\Omega} u^p v^q, \tag{1}$$

from which the centroid was obtained as

$$c_x = \frac{m_{10}}{m_{00}}, \qquad c_y = \frac{m_{01}}{m_{00}}. \tag{2}$$

The area feature used by the controller was the zeroth-order moment, $a = m_{00}$, which follows the standard moment-based visual servoing formulation [7].

The controller stored one target feature vector per object class. For symmetric shapes such as circles and regular polygons, the visual feature vector was

$$\mathbf{s} = \begin{bmatrix} x & y & \log(a/a^\star) \end{bmatrix}^\top, \tag{3}$$

where (x,y) are normalized image coordinates and $(\cdot)^\star$ denotes the desired feature value captured from a reference view. $\log(a/a^\star)$ is used instead of the raw area difference to compress the scale of the area error. Without this, large area discrepancies at distance would produce excessively high velocity commands, causing the arm to jerk toward the object rather than approach smoothly.

B. Visual Servoing Law

The normalized image coordinates were computed from the camera intrinsics as

$$x = \frac{c_x - u_0}{f}, \qquad y = \frac{c_y - v_0}{f}, \tag{4}$$

where (u_0, v_0) is the principal point and f is the focal length in pixels. Under the planar-object approximation used in image-based visual servoing, the feature error was written as

$$\mathbf{e} = \mathbf{s} - \mathbf{s}^\star, \tag{5}$$

and the camera velocity command was generated by

$$\mathbf{v}_c = -\lambda \mathbf{L}_s^+ \mathbf{e}, \tag{6}$$

where λ is a proportional gain, $\mathbf{L}_s$ is the interaction matrix, and $(\cdot)^+$ denotes the Moore-Penrose pseudoinverse [1]. The implementation used image-moment interaction rows for the centroid and area features together. The vertical velocity component was zeroed when the area error fell below a small threshold, and capped at a maximum value, to prevent jittery depth corrections near convergence.

Alignment was declared when the centroid error dropped below a pixel threshold and the area error dropped below a separate area threshold. These conditions were intentionally checked in image space rather than Cartesian space so that the stopping criterion remained consistent with the image-based controller. Once aligned, the perception module fit a minimum-area rectangle to the segmented contour. The short side of this rectangle provided an estimate of the required gripper width.

C. Robot Control and Task Sequencing

The robot-side process operated as a finite-state procedure with four stages: visual tracking, approach-and-grasp, perimeter search, and align-and-drop. During tracking, the ROS node requested fresh visual-servo commands over ZeroMQ and forwarded them directly as Cartesian twist commands in the tool frame, shown in Fig. 2.

A watchdog timer stopped the arm if a new command was not received within a short timeout, which avoided stale motion when communication or detection failed.

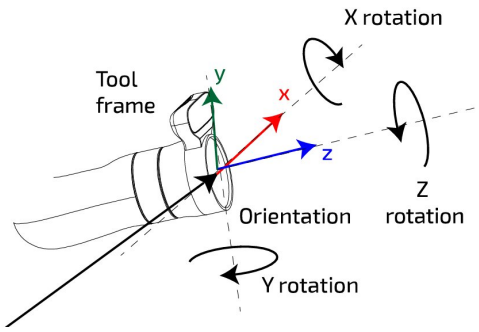

Fig. 2. The tool frame of the Kinova Gen3

When the alignment condition was met, the controller switched from continuous servoing to a predefined short forward approach, outlined in Algorithm 1.

Algorithm 1 Predefined grasp sequence

Require: alignment complete, gripper open

Move forward at v_{approach} for $T_{\text{approach}} = \frac{d_{\text{offset}}}{v_{\text{approach}}}$ seconds
Stop arm
Close gripper for $t_{\text{grasp}} \propto w_{\text{object}}$ seconds
Return to home pose

d_{offset} is the distance between the camera origin and the gripper in centimeters, and v_{approach} is the velocity of the forward motion in cm/s. The grasp duration was proportional to the short-side width of the object's bounding rectangle, clamped between 0.25 and 0.8 seconds to prevent under- or over-closing.

After grasping, the system searched for the drop-off bowl using a perimeter sweep over four predefined arm viewpoints that covered the workspace boundary, as outlined in Algorithm 2. The sweep consisted of four predefined joint angles at the corners of the workspace. At each viewpoint, the vision node was queried for bowl detection; the bowl was classified as a circle using the same YOLO model.

Algorithm 2 Perimeter sweep for bowl detection

Require: object grasped, bowl location unknown

Define corner viewpoints $P = \{P_1, P_2, P_3, P_4\}$
for each viewpoint $P_i \in P$ (cyclic) **do**
  Move arm to P_i
  Query vision node for bowl detection
  if bowl detected **then**
    Switch to visual servoing mode
    return
  end if
end for
Timeout after N attempts; halt

Detection of the bowl triggered a second alignment phase using the same visual-servo controller. Once aligned over the bowl, the robot executed a shortened forward motion (the drop sequence mirrored Algorithm 1, with the gripper opening instead of closing), released the object, and returned to the home pose.

IV. Experiments

To ensure the entire procedure remained consistent, we organized experiments into five stages: YOLO segmentation accuracy, visual servoing accuracy, grasping an object, a perimeter sweep across the workspace, and dropping an object. For each step, success was defined through reliability — that is, the number of successes within N trials. Where applicable, quantitative error measurements are also reported.

A. YOLO Detection & Segmentation

On the synthetic dataset, Fig. 3 shows the convergence of both metrics with the YOLO Segmentation model, with rapid improvement in the first 10 epochs before plateauing, with the achieved scores shown in Table I.

TABLE I
YOLO11 Segmentation model performance at epoch 50.

Metric	Box (B)	Mask (M)
Precision	0.984	0.980
Recall	0.978	0.975
mAP50	0.989	0.988
mAP50-95	0.977	0.939

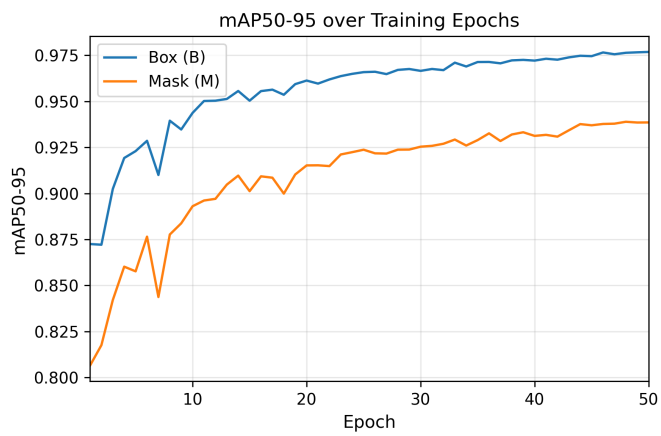

Fig. 3. Box and mask mAP50-95 over 50 training epochs.

Afterwards, we tested the trained YOLO model with a real-world camera on toy shapes, and the model was able to detect and segment the shapes under proper lighting conditions (minimal glare and shadows) with a confidence of 0.9. Fig. 4 shows the YOLO model accurately detecting and segmenting a toy cross.

B. Visual Servoing Accuracy

We defined the accuracy of our visual servoing task as the Euclidean distance of the centroid values from their ground truths, and the absolute difference of the area (in pixels):

$$e_{\text{dist}} = \sqrt{(c_x - c_x^*)^2 + (c_y - c_y^*)^2} \tag{7}$$

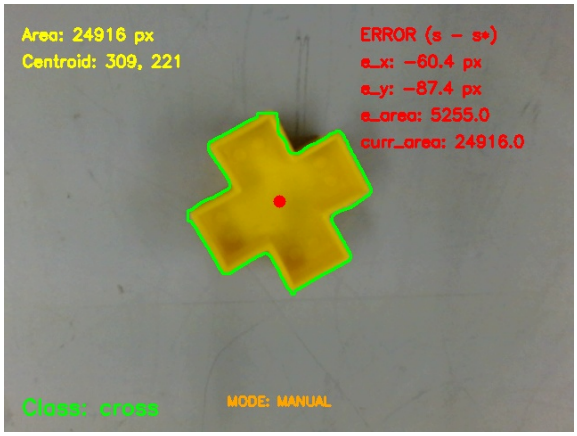

Fig. 4. YOLO segmentation output on a real toy cross, showing the detected mask contour.

$$e_{\text{area}} = |a - a^*| \tag{8}$$

where alignment was declared when $e_{\text{dist}} \leq \tau_{\text{dist}}$ and $e_{\text{area}} \leq \tau_{\text{area}}$.

Empirically, we found that $\tau_{\text{dist}} = 10$ and $\tau_{\text{area}} = 1500$ (in pixels) provided a balance of alignment precision and convergence time. Given any visible position in the workspace, the robot arm was able to align itself within the thresholded region in under 10 seconds. With 10 trials, the Kinova successfully aligned to the target object in all 10 trials (under controlled lighting and consistent frame rate). There were near-failures — defined as taking longer than 8 seconds to align itself — for all four shapes, which were caused by lighting variation (large shadows and glare), poor camera resolution, or an inconsistent frame rate.

C. Object Grasping

The grasp sequence (Algorithm 1) was tested with four different shapes — pentagon, hexagon, cross, ellipse — throughout 10 trials. The pentagon, hexagon, and cross were successfully grasped in all 10 trials, whereas the ellipse had 6 successful trials. Varying the orientation of the ellipse caused the width to be miscalculated, affecting the gripper's closing duration. In all four failures, the gripper did not close sufficiently to secure the ellipse.

D. Perimeter Sweep

The perimeter sweep (Algorithm 2) was tested over 10 trials, where the bowl's position was varied — the bowl was placed at different corners and introduced at varying points during the sweep (after 1, 2, or 3 full cycles). The robot detected the bowl within 4 viewpoints in all 10 trials. The timeout mechanism was also verified: when no bowl was present, the robot halted after $N = 5$ full-sweep cycles.

E. Dropping the Object

Dropping the object required locating the bowl via visual servoing (Section III-B), aligning over it, and releasing the object. To simplify bowl detection, the bowl was classified as a circle using the same YOLO model. Among the 10 trials, 9 were successful. The failure was likely caused by glare on the bowl, causing the bowl's segmentation mask to jitter, which caused the area measurement to vary. This led to one of two outcomes: the robot was slow to align with the bowl, or it failed to align within 10 seconds.

Overall, the full pick-and-place procedure took approximately 1 minute and 40 seconds on average across N=5 trials.

V. Conclusion

This project demonstrated a complete vision-guided pick-and-place system combining YOLO-based instance segmentation with moment-based visual servoing on a Kinova Gen3 robot arm. The system achieved a 9/10 success rate on the full pipeline, with each trial completing in approximately 1 minute and 40 seconds. Individual stages performed reliably: the YOLO model achieved a mask mAP50 of 0.988 on synthetic data and transferred to real objects with ≥ 0.8 confidence, visual servoing aligned to target objects in all 10 trials, and grasping succeeded for the shapes tested.

The primary limitations were sensitivity to lighting conditions and difficulty handling asymmetric objects. Glare and shadows caused inconsistent segmentation masks, degrading servoing stability, and orientation-dependent shapes led to grasp failures. Future work could address these issues by incorporating the moment-based orientation feature α, and improving robustness to lighting variation through synthetic lighting perturbations.

References

[1] F. Chaumette and S. Hutchinson, "Visual servo control. I. Basic approaches," IEEE Robot. Autom. Mag., vol. 13, no. 4, pp. 82–90, Dec. 2006.

[2] O. Tahri and F. Chaumette, "Point-based and region-based image moments for visual servoing of planar objects," IEEE Trans. Robot., vol. 21, no. 6, pp. 1116–1127, Dec. 2005.

[3] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," GitHub repository, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[4] C. E. Smith and N. P. Papanikolopoulos, "Vision-guided robotic grasping: Issues and experiments," in Proc. IEEE Int. Conf. Robot. Autom. (ICRA), Minneapolis, MN, USA, 1996, vol. 4, pp. 3203–3208.

[5] Kinova Robotics, "Kinova Kortex API documentation." [Online]. Available: https://docs.kinovarobotics.com/ Accessed: Mar. 18, 2026.

[6] G. Bradski, "The OpenCV library," Dr. Dobb's Journal: Software Tools for the Professional Programmer, vol. 25, no. 11, pp. 120–123, 2000.

[7] F. Chaumette, "Image moments: a general and useful set of features for visual servoing," IEEE Trans. Robot., vol. 20, no. 4, pp. 713–723, Aug. 2004.